As we can see, null bytes do not present any problems but check the output when we used `strpbrk()` with some multi-byte (UTF-8) characters. Although we specified U+141 (uppercase "Ł") for our search pattern, the function returns from the very beginning of the input (lowercase "ł"). Can you work out why? The reason is that both U+0141 and U+0142 are UTF-8 encoded as *0xC5 0x81* and *0xC5 0x82*, respectively. `strpbrk()` tests the very first byte of the `$ship` string against our search bytes (*0xC5* and *0x81*); it matches *0xC5*, and so we get the entire input string returned. There is no `mbstring` equivalent of this function, but the same results can be achieved with a multi-byte safe regular expression, as we shall see shortly.

strtok()

Our final sub-string extractor function is `strtok()`, which is quite specialized. `strtok()` is used to split strings into tokens and is not often used outside of the context it is designed for. If you have been programming for a while, you'll probably have at least encountered the term "tokenization"; it is one of the processes involved in compiling source code into something executable.

To a compiler or interpreter (compilers produce independently executable artifacts such as .exe files, interpreters compile *and* execute in one go – PHP binaries are interpreters), the source code of a program is initially just one long string of text. This string must first be parsed for correctness before it can be transformed into lower-level instructions that a machine can follow, and the first step in parsing is lexical analysis: the string must be broken into sub-units; we call these sub-units tokens. We briefly encountered these concepts earlier in the book.

PHP offers user-space constants for all of its internal token values (identifiers such as keywords or variable names are `T_STRING`, the object operator `->` is `T_OBJECT_OPERATOR`, and so on), and I encourage you to familiarize yourself with them here: https://www.php.net/manual/en/tokens.php. However, demonstrating `strtok()` by parsing PHP itself is probably not where we should start, so let's go with something simpler.

```
$text = <<<EOT
    Hi there, and
    welcome to tokens!
    EOT;
$delimiters = " \n";
```